

Table des matières

Le poste de travail du parcours — guide d'installation	1
Comment ce guide fonctionne.....	2
Les trois terminaux, et ce que chacun sert	3
Ce que vous installez, et pourquoi.....	4
Les versions de référence	5
Étape 1 — Créer le dossier du projet.....	6
Étape 2 — Installer Python, avec conda	6
Étape 3 — Installer Git	7
Étape 4 — Installer DuckDB	7
Étape 5 — Installer Ubuntu dans WSL 2, puis Docker Engine	7
Étape 6 — Régler l'arrêt automatique d'Ubuntu.....	9
Étape 7 — Installer le pilote ODBC de PostgreSQL	10
Étape 8 — Installer PostgreSQL sur le poste.....	10
Étape 9 — Générer les données de Boréal	11
Récapitulatif	13
Dépannage	14
Annexe — Tout le code, d'un seul tenant.....	15

Le poste de travail du parcours — guide d'installation

Commun aux sept formations du domaine des données et à ses deux fondations

Ce que vous allez faire. Monter une fois le poste que tout le parcours emploie : Python et son environnement, Git, DuckDB, Docker dans l'Ubuntu de WSL 2, le pilote ODBC de PostgreSQL, et les données de Véhicules Boréal inc. Comptez une à deux heures, estimation faite surtout de téléchargements et d'installateurs qui tournent seuls. Vous n'installerez plus rien d'un bloc ensuite : chaque cours qui allume un service le fait au moment où il s'en sert, en quelques commandes, et ce guide reste la référence à laquelle il renvoie.

La liste de contrôle du socle :

- ☐ Dossier `Projet_Boreal/` créé dans `Data_Domain`
- ☐ Environnement `conda` formations actif ; `import duckdb, pyodbc, pytest` qui répond
- ☐ `git --version` qui répond, `user.name` et `user.email` réglés
- ☐ `duckdb --version` qui répond dans un terminal Windows
- ☐ Ubuntu dans WSL 2, Docker Engine installé, `docker run hello-world` qui affiche `Hello from Docker!`
- ☐ Pilote `psqlODBC` installé : PostgreSQL Unicode(x64) dans la liste des pilotes ODBC
- ☐ Données de Boréal générées dans `Projet_Boreal/sources/` : 1 763 fichiers, empreinte globale `37521016091c44da...`

Comment ce guide fonctionne

Le parcours partage une seule plateforme de données et un seul poste de travail. Ce guide les monte une fois, pour les sept formations et les deux fondations. Rien ici n'est le travail d'un métier de la donnée : en entreprise, les TI montent le poste, l'équipe des systèmes livre la plateforme, et les systèmes sources tournent déjà. On le fait ici parce qu'il faut bien une machine à qui parler.

Ce que ce guide monte, on l'appelle le socle : ce que presque tout le parcours emploie. Ce qu'il ne monte pas, ce sont les services qu'une seule formation allume, et qui coûtent cher à laisser dormir : la base de l'ERP de Boréal, Airflow, Spark, Kafka, une base de séries temporelles. Chacun s'installe au début du cours qui s'en sert, dans un bloc d'ouverture de quelques commandes qui renvoie ici pour le reste.

Pourquoi — ne pas tout monter d'avance. Monter toute la pile avant le premier cours met trois à cinq heures de téléchargement et de configuration entre vous et votre premier geste de métier, et vous fait installer au premier jour des services que le neuvième cours vous expliquera. Vous les auriez oubliés avant de vous en servir. La règle du parcours est l'inverse : le socle maintenant, l'outil le jour où il sert.

Étape	Ce qu'elle installe	Qui en a besoin
1	le dossier <code>Projet_Boreal/</code>	tout le parcours
2	Python, par <code>conda</code>	tout le parcours
3	Git	tout le parcours
4	DuckDB, le paquet et la ligne de commande	tout le parcours
5	Ubuntu dans WSL 2, puis Docker Engine	tout le parcours, sauf Architecture et Gouvernance

Étape	Ce qu'elle installe	Qui en a besoin
6	le réglage d'arrêt automatique d'Ubuntu	avec l'étape 5
7	le pilote ODBC de PostgreSQL	Data Engineering
8	PostgreSQL sur le poste	la fondation SQL avancé
9	les données de Boréal	les sept formations du parcours

Les étapes 7 et 8 se sautent tant que la formation qui les demande n'est pas commencée. Les autres se font dans l'ordre : chacune vérifie la précédente.

Ce que chaque formation ajoute au socle, et où :

Formation	Ce qu'elle allume en plus	Où
Data Engineering	la base de l'ERP de Boréal, Airflow, puis Spark, Kafka, TimescaleDB, Grafana	au bloc d'ouverture du cours qui s'en sert
Analytics Engineering	dbt sur DuckDB	à son premier cours
Business Intelligence	Power BI Desktop	au cours qui construit le rapport
Data Science, ML Engineering	scikit-learn, MLflow	à leur premier cours
AI Engineering	Ollama et un modèle local	à son premier cours
Architecture et Gouvernance	rien : de quoi écrire et dessiner	sans objet
SQL avancé	PostgreSQL et AdventureWorks	étape 8 de ce guide, puis son Cours 00
Snowflake	un compte d'essai en édition Enterprise, rien à installer	à son Cours 00

Le README.md du parcours dit ce que chacun de ces outils coûte et ce qu'on perd à le sauter.

Les trois terminaux, et ce que chacun sert

Le parcours fait travailler dans trois terminaux. Un cours dit toujours dans lequel vous êtes ; ce qui suit dit une fois pour toutes pourquoi il y en a trois, pour que vous n'ayez plus à changer de fenêtre sur ordre sans comprendre.

Terminal	Ce qu'on y tape	Comment on l'ouvre
l'Anaconda Prompt	Python, pip, Git, les générateurs, psql	menu Démarrer, <i>Anaconda Prompt</i>
la ligne de commande DuckDB	le SQL de vérification	duckdb dans l'Anaconda Prompt
Ubuntu, dans WSL 2	tout ce qui touche Docker	ws1 dans l'Anaconda Prompt, ou <i>Ubuntu</i> au menu Démarrer

La raison de fond tient en une phrase : Docker Engine vit dans Ubuntu, et pas sous Windows. Tout ce qui parle à Docker (les services en conteneurs, Airflow, Kafka) se tape donc dans Ubuntu, et le reste dans l'Anaconda Prompt. La ligne de commande DuckDB, elle, n'est pas un terminal du système : c'est un client SQL qui s'ouvre dans le terminal où vous êtes, et qui attend du SQL. On en sort par `.quit`.

Ce que vous installez, et pourquoi

Pièce	Son rôle	Pourquoi celle-là
conda, environnement <i>formations</i>	isoler Python et ses paquets	le même environnement sert toutes les formations de l'atelier ; on le crée une fois
Git	versionner le code	un projet qu'on ne peut pas reconstruire depuis son dépôt n'est pas un livrable, et chaque formation en ouvre un
DuckDB	interroger des fichiers en SQL, sans serveur	il tient sur un portable, lit CSV, JSON et Parquet sur place, et se charge comme une bibliothèque Python
WSL 2 et Docker	faire tourner les services dans des conteneurs Linux	installer ces services directement sous Windows coûte des heures sans rien apprendre, et ni Airflow ni Kafka n'y tournent ; un conteneur se jette et se recrée en une commande
psqlODBC	laisser un programme Windows lire une base PostgreSQL	Data Engineering lit la base de l'ERP comme on lit une base de production, par un pilote
PostgreSQL	un moteur transactionnel sur le poste	la fondation SQL avancé y travaille son second terrain, celui des bases qui font tourner l'entreprise

Pièce	Son rôle	Pourquoi celle-là
les données de Boréal	la matière de tout le parcours	elles se fabriquent, elles ne se téléchargent pas : un générateur à tirage fixé donne à tout le monde les mêmes octets

WSL 2 (*Windows Subsystem for Linux*) fait tourner un vrai Linux, ici Ubuntu, à côté de Windows, sans machine virtuelle à gérer soi-même. Docker Engine s’y installe comme sur un serveur, avec les commandes de sa documentation officielle : c’est la voie du parcours. Depuis Ubuntu, les fichiers de Windows restent visibles sous `/mnt/c/`.

Pourquoi — Docker Engine plutôt que Docker Desktop. Docker Desktop, l’application graphique de Docker, fait le même travail sur Windows 11 et sur macOS, et les commandes des labos y sont identiques. Le parcours ne le prend pas comme référence : Docker ne le soutient plus sur Windows 10, sorti du calendrier de service de Microsoft en octobre 2025, et sa gratuité est réservée aux particuliers et aux petites entreprises. Docker Engine, dans Ubuntu, est gratuit sans condition et se comporte comme sur le serveur Linux où vos conteneurs finiront.

Tout se range dans le dossier du parcours :

Data_Domain/	le dossier du parcours, dans le dépôt des formations
├─ univers/	ce que partagent les formations du parcours
│ ├─ boreal/generer_boreal.py	le générateur des données de Boréal
│ └─ plateforme/	les fichiers des services, reçus tout prêts
├─ 01_Data_Engineering/	les supports d'une formation
└─ Projet_Boreal/	votre projet, que le dépôt des formations ignore
├─ sources/	ce que les systèmes de Boréal livrent (étape 9)
├─ plateforme/	les services, quand un cours en allume un
└─ 01_de/	votre dépôt Git, créé au premier cours de Data Engineering

Les versions de référence

Les chiffres et les sorties que les cours affichent ont été pris avec ces versions, relevées le 2026-09-17. Une version plus récente convient presque toujours ; si une sortie diffère de ce qu’un cours annonce, c’est la première chose à regarder. Les `requirements.txt` des formations bornent chaque paquet sous sa prochaine version majeure, `duckdb>=1.5, <2` : une version majeure peut changer ce qu’une commande affiche, et remonter une borne demande de remesurer les chiffres des cours.

Outil	Version de référence	Relevé le
Python	3.11	2026-09-17
DuckDB	1.5.5, publiée le 2026-07-22	2026-09-17
pyodbc et pytest	5.3.0 et 9.1.1	2026-09-22

Outil	Version de référence	Relevé le
dbt-core et dbt-duckdb	1.12.3 et 1.11.0	2026-09-22
Ubuntu	26.04 LTS ; la 24.04 convient aussi	2026-09-17
psqlODBC	REL-18_00_0004	2026-09-17
PostgreSQL, sur le poste	18	2026-09-17
PostgreSQL, en conteneur	image postgres:18 pour l'ERP de Boréal, postgres:16 pour la base de métadonnées d'Airflow	2026-09-17
Airflow	3.3.1, publiée le 2026-08-12	2026-09-17

Étape 1 — Créer le dossier du projet

Tout ce qui tourne pour Boréal tient dans un seul dossier, `Projet_Boreal/`. Il se range dans le dossier du parcours, `Data_Domain/`, à côté des dossiers des formations et du dossier `_univers/`. Ouvrez l'Anaconda Prompt, placez-vous dans `Data_Domain`, puis :

```
mkdir Projet_Boreal
```

`Projet_Boreal/` est votre plateforme de données : les données que les systèmes de Boréal livrent, les services qui tournent pour elle, puis le lac et l'entrepôt que vous bâtirez. Il n'est pas versionné : le dépôt des formations l'ignore, et tout ce qu'il contient se reconstruit. Vos dépôts de code s'y ajouteront, un par formation : les créer est le premier geste du métier, pas de l'installation.

Sauf mention contraire, l'Anaconda Prompt de ce guide travaille depuis `Data_Domain`.

Étape 2 — Installer Python, avec conda

L'atelier utilise un seul environnement conda, `formations`, partagé par toutes ses formations. Si vous l'avez déjà, activez-le. Sinon, créez-le une fois :

```
conda create -n formations python=3.11 -y
conda activate formations
```

L'invite affiche (`formations`) au début de la ligne. C'est le signe que vous êtes dans le bon environnement, et chaque formation vous demandera de le vérifier.

Les paquets Python d'une formation tiennent dans son fichier `requirements.txt`. Celui de Data Engineering installe DuckDB, pyodbc qui lit la base de l'ERP par ODBC, et pytest qui lance les tests du pipeline. Installez-les maintenant si vous commencez par cette formation :

```
pip install -r 01_Data_Engineering\requirements.txt
python -c "import duckdb, pyodbc, pytest; print(duckdb.__version__)"
```

La seconde ligne ne doit rien afficher d'autre qu'un numéro de version : une erreur `ModuleNotFoundError` voudrait dire qu'un paquet manque, ou que formations n'est pas actif.

Étape 3 — Installer Git

Git versionne votre code. Chaque formation du parcours ouvre un dépôt dès son premier cours, et une formation entière lui est consacrée dans l'atelier.

Dans l'Anaconda Prompt :

```
git --version
```

S'il répond un numéro de version, passez au réglage de l'identité. Sinon, installez-le depuis <https://git-scm.com/download/win> (l'installateur 64 bits), en acceptant les choix par défaut, puis fermez et rouvrez l'Anaconda Prompt avant de revérifier.

Git refuse d'enregistrer une modification tant qu'il ne sait pas qui la signe. Réglez-le une fois, pour tous vos dépôts :

```
git config --global user.name "Votre Nom"
git config --global user.email "vous@exemple.com"
git config --global --list
```

La dernière ligne doit montrer vos deux valeurs. Sans elles, le premier `git commit` d'un labo s'arrête sur `Author identity unknown`.

Étape 4 — Installer DuckDB

DuckDB s'installe deux fois, pour deux usages. Le paquet Python, posé à l'étape 2, sert au code qui interroge l'entrepôt. La ligne de commande DuckDB est le client SQL des labos : on y tape les requêtes de vérification. Elle s'installe par winget, le gestionnaire de paquets de Windows :

```
winget install DuckDB.cli
```

winget ajoute la commande `duckdb` au chemin de recherche de Windows, mais seul un terminal ouvert après l'installation la voit. Fermez l'Anaconda Prompt, rouvrez-le, réactivez formations, revenez dans Data_Domain, puis vérifiez :

```
duckdb --version
```

Python affiche 1.5.5, la ligne de commande `v1.5.5 (Variegata)` suivi d'un identifiant. DuckDB ne demande ni service, ni port, ni mot de passe. Sous Windows, il a seulement besoin du Microsoft Visual C++ Redistributable, déjà présent sur la plupart des postes.

Étape 5 — Installer Ubuntu dans WSL 2, puis Docker Engine

Docker fait tourner les services du parcours. Il s'installe en trois temps : Ubuntu dans WSL 2, le dépôt de paquets de Docker, puis Docker Engine lui-même.

D'abord, ce que le poste a déjà. Dans l'Anaconda Prompt :

```
wsl -l -v
```

Si la liste montre une distribution Ubuntu en version 2, passez au paragraphe suivant. Sinon, ouvrez un terminal Windows en administrateur et installez-la :

```
wsl --install -d Ubuntu-26.04
```

L'installation peut demander un redémarrage. Au premier lancement, Ubuntu vous demande un nom d'utilisateur et un mot de passe : ce sont ceux de Linux, sans lien avec votre compte Windows, et sudo vous redemandera ce mot de passe.

Ouvrez Ubuntu : tapez wsl dans l'Anaconda Prompt, ou lancez Ubuntu depuis le menu Démarrer. L'invite change, vous êtes dans Linux. Vérifiez que systemd, le gestionnaire de services de Linux, est bien le premier processus :

```
ps -p 1 -o comm=
```

La réponse attendue est systemd. C'est lui qui démarrera Docker chaque fois qu'Ubuntu démarre. S'il répond autre chose, voyez le dépannage.

Ensuite, le dépôt de paquets de Docker, puis Docker Engine. Ces commandes sont celles de la documentation officielle de Docker pour Ubuntu (docs.docker.com/engine/install/ubuntu) :

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

sudo demande votre mot de passe Linux au premier usage. Le bloc du milieu écrit un fichier qui déclare le dépôt de Docker : collez-le d'un seul tenant, de sudo tee jusqu'au second EOF. apt demande une confirmation avant chaque installation.

Dernier réglage : sans lui, chaque commande docker demanderait sudo. Ajoutez votre utilisateur au groupe docker, puis fermez Ubuntu pour que le changement prenne effet :

```
sudo usermod -aG docker $USER
exit
```

Rouvrez Ubuntu (wsl), puis :

```
docker --version
docker compose version
docker run hello-world
```

Les deux premières lignes affichent les versions du moteur et de compose, l'outil qui démarre plusieurs services d'un coup. La troisième télécharge une image depuis le registre public de Docker, en crée un conteneur et l'exécute : il affiche un texte qui commence par Hello from Docker!, puis s'arrête. Ce conteneur arrêté existe encore. Listez-le, puis faites le ménage :

```
docker ps -a
docker container prune
```

docker ps -a montre tous les conteneurs, y compris ceux qui sont arrêtés ; docker container prune supprime les arrêtés (confirmez par y). L'image, elle, reste en cache : un prochain docker run hello-world ne la retéléchargera pas. Quittez Ubuntu avec exit. Ce que sont une image et un conteneur, et pourquoi on ne modifie jamais un conteneur à la main, s'apprend dans le cours qui vous fera en construire un.

Astuce — rendre la mémoire. Docker démarre avec Ubuntu, par systemd, et WSL peut réserver jusqu'à la moitié de la mémoire de la machine. Quand vous avez fini, wsl --shutdown, tapé dans l'Anaconda Prompt, arrête Ubuntu et ses conteneurs d'un coup.

Astuce — Docker Desktop sur Windows 11 ou sur macOS. Si vous avez déjà Docker Desktop, gardez-le : docker run hello-world et toutes les commandes des labos s'y tapent pareil, dans votre terminal habituel. Le début de ce guide dit pourquoi le parcours ne le prend pas comme référence.

Piège — Docker Desktop et Docker Engine sur le même Windows. Si Docker Desktop est resté installé sur le poste où vous venez de monter Docker Engine, deux moteurs cohabitent, chacun avec ses images et ses conteneurs. Dans Ubuntu, docker parle à Docker Engine ; dans l'Anaconda Prompt, docker parle à Docker Desktop. Un conteneur lancé du mauvais côté est invisible de l'autre, et deux conteneurs qui publient le même port se le disputent. Tapez les commandes docker des labos dans Ubuntu. En cas de doute, docker info le dit à sa ligne Operating System : Ubuntu 26.04 LTS pour Docker Engine, Docker Desktop pour l'autre. Le plus simple reste de désinstaller Docker Desktop (Paramètres de Windows, Applications).

Piège — VirtualBox et WSL 2 sur la même machine. Si vous utilisez VirtualBox (formation Linux), sachez que WSL 2 active la plateforme de virtualisation de Windows. Les versions récentes de VirtualBox savent cohabiter avec elle, mais leurs machines virtuelles peuvent tourner plus lentement. Ce n'est pas une panne.

Étape 6 — Régler l'arrêt automatique d'Ubuntu

WSL arrête Ubuntu quelques secondes après la fermeture de votre dernier terminal Ubuntu, et les conteneurs s'arrêtent avec elle. En cours de labo, cela veut dire une base qui disparaît parce que vous avez fermé une fenêtre. Le délai se règle dans %UserProfile%\wslconfig, un fichier que Windows ne crée pas tout seul. Dans l'Anaconda Prompt :

```
notepad %UserProfile%\wslconfig
```

Le Bloc-notes propose de créer le fichier s'il n'existe pas. Écrivez-y ces deux lignes, enregistrez, fermez, puis `wsl --shutdown` pour que le réglage soit relu au prochain démarrage :

```
[general]
instanceIdleTimeout=3600000
```

La valeur est en millisecondes : une heure, au lieu des 15 secondes par défaut. La valeur -1 existe et ne l'arrête jamais, mais elle garde aussi vos conteneurs allumés jusqu'au redémarrage du poste, ce qui n'est pas ce qu'on veut sur un portable.

Piège — le réglage voisin, qui ne règle pas ce délai. `vmIdleTimeout`, qu'on croise souvent dans les réponses en ligne, arrête la machine virtuelle entière et n'existe que sur Windows 11. C'est `instanceIdleTimeout`, dans la section `[general]`, qui commande l'arrêt d'une distribution inactive. Réglé le 2026-09-17 d'après la documentation de WSL ([learn.microsoft.com](https://learn.microsoft.com/en-us/windows/wsl/reference), tableau « General WSL settings ») ; le comportement sous Windows 10 reste à vérifier sur le terrain, et en attendant, gardez une fenêtre Ubuntu ouverte pendant un labo qui fait tourner des conteneurs.

Étape 7 — Installer le pilote ODBC de PostgreSQL

Pour la formation Data Engineering, qui lit la base de l'ERP de Boréal depuis Windows.

Le pilote `psqlODBC` s'installe une fois, avec les droits d'administrateur. Téléchargez l'installateur 64 bits depuis la page des versions de `psqlODBC`, <https://www.postgresql.org/ftp/odbc/releases/> : la dernière version est `REL-18_00_0004`, et son fichier `psqlodbc_x64.msi`. Lancez-le et acceptez les choix par défaut.

Puis, dans l'Anaconda Prompt, formations actif :

```
python -c "import pyodbc; print(pyodbc.drivers())"
```

La liste des pilotes ODBC du poste doit contenir un pilote dont le nom commence par `PostgreSQL Unicode`. Sur le poste de l'auteur, quatre entrées `PostgreSQL` apparaissent, deux jeux de caractères pour deux architectures :

```
['PostgreSQL ANSI(x64)', 'PostgreSQL ANSI', 'PostgreSQL Unicode(x64)', 'PostgreSQL Unicode']
```

C'est `PostgreSQL Unicode(x64)` qui sert : `Unicode` parce que les libellés de l'ERP portent des accents, et 64 bits parce que le Python de formations l'est. Si aucune entrée n'apparaît, le pilote n'est pas installé, ou vous avez pris la version 32 bits.

Étape 8 — Installer PostgreSQL sur le poste

Pour la fondation SQL avancé, qui y monte sa base transactionnelle. Le parcours lui-même n'en a pas besoin : ses bases PostgreSQL tournent en conteneurs.

PostgreSQL s'installe sous Windows par l'installateur d'EDB, <https://www.postgresql.org/download/windows/>. Trois choses à surveiller pendant l'installation :

- laissez le port 5432 ;
- choisissez un mot de passe pour l'utilisateur postgres, et notez-le : rien ne le redemandera, et tout le reste en dépend ;
- gardez pgAdmin 4, l'interface graphique, et surtout les outils en ligne de commande.

À la fin, l'installateur propose *Stack Builder*, des extensions facultatives : cliquez sur *Cancel*. PostgreSQL tourne dès lors comme un service Windows, et démarre avec la machine.

L'installateur d'EDB n'ajoute pas les outils en ligne de commande au chemin de recherche de Windows. Ajoutez-les, en remplaçant 18 par votre version, puis fermez et rouvrez l'Anaconda Prompt :

```
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path","User") + ";C:\Program Files\PostgreSQL\18\bin", "User")
```

Cette commande se tape dans PowerShell, pas dans l'Anaconda Prompt : ouvrez *Windows PowerShell* au menu Démarrer, collez-la, puis revenez à l'Anaconda Prompt pour la suite. Vérifiez :

```
psql --version
psql -U postgres -c "SELECT version();" 
```

La première répond psql (PostgreSQL) 18.6, ou un numéro voisin. La seconde demande le mot de passe de postgres : tapez-le, rien ne s'affiche pendant la frappe, c'est normal. Elle répond une longue ligne qui commence par PostgreSQL 18.6 et décrit la construction du serveur. Si elle répond au lieu de cela `password authentication failed`, c'est le mot de passe choisi à l'installation qui ne concorde pas.

Piège — pgAdmin ne remplace pas le chemin de recherche. pgAdmin se passe de ligne de commande pour écrire des requêtes, et c'est un bon client. Mais le chargement d'un jeu de données passe par `createdb` et `psql`, qui sont des programmes : sans le réglage ci-dessus, ils répondent « n'est pas reconnu », quel que soit le client que vous employez ensuite.

Étape 9 — Générer les données de Boréal

Boréal n'a pas encore de données : on les fabrique. Le générateur `_univers/boreal/generer_boreal.py`, livré avec le dépôt des formations, produit ce qu'un ingénieur de données recevrait réellement des systèmes de Boréal. Il n'a besoin que de la bibliothèque standard de Python.

Il vit dans `_univers/`, un dossier que partagent les sept formations du parcours : c'est un outil, pas une pièce de votre projet, et on ne le copie nulle part. Dans l'Anaconda Prompt, depuis `Data_Domain` :

```
python _univers/boreal/generer_boreal.py --projet Projet_Boreal
```

Comptez un peu plus d’une minute. --projet désigne le dossier du projet, qui doit exister : le générateur ne le crée pas, et s’arrête avec un message clair s’il ne le trouve pas. Il écrit dans `Projet_Boreal/sources/`, qu’il refait de zéro, puis affiche son bilan :

```
Génération de Véhicules Boréal inc., du 2022-01-01 au 2025-12-31...
meteo/      11 stations, 17061 jours, 37068 heures
erp/        48 extraits mensuels (et 2023-03_bis), 25 483 commandes, 77 732 postes, 27 775 véhicules à l'état du 2025-12-31
usine/      historien 92 jours, 275 436 lignes, 58 étiquettes ; 779 bons de travail ; 140 255 relevés du compteur
garanties/   48 extraits mensuels : 26 732 enregistrements, 53 865 visites, 5 623 lignes de réclamations
telemetrie/ 884 jours, 158 146 messages reçus de 6 759 véhicules, dont 2 303 doublons
corpus/     33 documents publiés sur 34 au catalogue, 16 126 mots

Écrit dans ...\\Projet_Boreal\\sources
1 763 fichiers, 138,4 Mo, empreinte globale 37521016091c44da...
Durée : 74.4 s.
Conforme à la référence des cours (la génération par défaut).
```

Tout le monde obtient exactement les mêmes fichiers : le tirage est fixé, et la dernière ligne vous le dit. Le générateur compare lui-même son empreinte globale à celle que le dépôt versionne, dans `_univers/boreal/REFERENCES.json` : « conforme » veut dire que vos 1 763 fichiers sont identiques, octet pour octet, à ceux de l’auteur ; seule la durée dépend de votre machine. S’il répond « DIFFÉRENTE », il donne les deux empreintes et vous renvoie à `sources/empreintes.sha256`, qui porte celle de chaque fichier. Relancer le générateur redonne la même empreinte, parce qu’il refait `sources/` de zéro à chaque fois.

Le contrôle ne vaut que pour les générations que les cours citent : celle-ci, celle du Labo 06 (--jusqua 2026-03) et celle du Labo 07 (--jusqua 2026-03 --historien complet). Une autre période se génère très bien, mais rien ne l’a mesurée, et le générateur se tait.

Dossier ou fichier de sources/	Ce qu’il contient
LISEZMOI.md, dictionnaire_boreal.md, empreintes.sha256	la table des matières, le sens de chaque fichier et de chaque colonne, l’empreinte de chaque fichier
erp/	extraits/AAAA-MM/, un CSV par table SAP et par mois, de 2022-01 à 2025-12 ; postgresql/, la copie de la base
usine/	historien/, un CSV par jour, du 2025-10-01 au 2025-12-31 ; referentiel/, gmao/, compteur/
garanties/	un dossier par mois : les extraits du portail des garanties
telemetrie/	sorties_AAAA-MM-JJ.jsonl, un fichier par jour depuis le 2023-08-01
corpus/	la documentation, rangée par famille, et son catalogue index.csv

Dossier ou fichier de sources/	Ce qu'il contient
meteo/	l'instantané des observations d'Environnement et Changement climatique Canada

On ne modifie jamais ces fichiers : ils sont la référence, comme un export reçu d'un autre service.

Ces fichiers ne vont dans aucun dépôt Git. Ils simulent la sortie de systèmes externes, et l'on ne versionne pas des données brutes : elles pèsent 138 Mo et se régénèrent à l'identique. Elles vivent à côté de vos dépôts, pas dedans, et le dépôt des formations ignore `Projet_Boreal/` en entier. Le générateur, lui, est versionné avec les formations.

Récapitulatif

Vous avez...	Preuve
créé le dossier du projet	<code>Projet_Boreal/</code> existe dans <code>Data_Domain</code>
installé Python	<code>(formations) actif; import duckdb, pyodbc, pytest</code> passe
installé Git	<code>git --version</code> répond, et <code>git config --global --list</code> montre votre nom et votre courriel
installé DuckDB	<code>duckdb --version</code> répond <code>v1.5.5</code> , ou une version plus récente
installé Docker	dans Ubuntu, <code>docker run hello-world</code> affiche <code>Hello from Docker!</code>
réglé l'arrêt d'Ubuntu	<code>%UserProfile%\wslconfig</code> contient <code>instanceIdleTimeout</code>
installé le pilote ODBC	<code>pyodbc.drivers()</code> contient <code>PostgreSQL Unicode(x64)</code>
installé PostgreSQL	<code>psql -U postgres -c "SELECT version();" répond PostgreSQL 18.6...</code>
généralisé les données de Boréal	1 763 fichiers dans <code>sources/</code> , empreinte globale <code>37521016091c44da...</code>

Le poste est prêt. Ouvrez le `README.md` du parcours pour choisir par où entrer, puis la mise en situation de la formation que vous commencez : elle dit pour qui vous travaillez avant que vous tapiez une ligne. Chaque cours qui a besoin d'un service de plus l'allume lui-même, en tête de son labo, et renvoie ici pour le reste.

Dépannage

Symptôme	Cause probable	Correctif
'conda' n'est pas reconnu en tant que commande interne	un terminal ordinaire au lieu de l'Anaconda Prompt	ouvrir l'Anaconda Prompt
ERROR: Could not open requirements file	l'Anaconda Prompt n'est pas dans Data_Domain	se placer dans Data_Domain, puis relancer l'installation des paquets
ModuleNotFoundError: No module named 'duckdb' (ou pyodbc, pytest)	l'environnement formations n'est pas actif, ou les paquets n'y sont pas installés	conda activate formations, puis l'étape 2
'git' n'est pas reconnu en tant que commande interne	Git n'est pas installé, ou le terminal a été ouvert avant l'installation	installer Git, puis fermer et rouvrir l'Anaconda Prompt
Author identity unknown au premier commit d'un labo	user.name et user.email ne sont pas réglés	les deux git config --global de l'étape 3
'duckdb' n'est pas reconnu en tant que commande interne	la ligne de commande DuckDB n'est pas installée, ou l'Anaconda Prompt a été ouvert avant son installation	winget install DuckDB.cli, puis fermer et rouvrir l'Anaconda Prompt
wsl --install refuse de s'exécuter	le terminal n'est pas ouvert en administrateur	rouvrir le terminal en administrateur (clic droit, <i>Exécuter en tant qu'administrateur</i>)
L'installation d'Ubuntu échoue avec l'erreur 0x80370102	la virtualisation matérielle est coupée	l'activer dans le BIOS ou l'UEFI (Intel VT-x ou AMD-V), puis redémarrer
WSL réclame une mise à jour	WSL trop ancien	dans un terminal en administrateur : wsl --update, puis redémarrer
ps -p 1 -o comm= ne répond pas systemd	systemd n'est pas activé dans Ubuntu	dans Ubuntu, ajouter à /etc/wsl.conf les deux lignes [boot] et systemd=true (sudo nano /etc/wsl.conf), puis wsl --shutdown dans l'Anaconda Prompt, et rouvrir Ubuntu
E: Unable to locate package docker-ce	le dépôt de Docker n'est pas déclaré, ou apt update n'a pas tourné après	refaire le bloc qui commence par sudo tee, puis sudo apt update, puis l'installation

Symptôme	Cause probable	Correctif
permission denied while trying to connect to the Docker daemon socket	le groupe docker n'est pas encore actif dans ce terminal	exit, rouvrir Ubuntu ; si l'erreur reste, wsl -- shutdown dans l'Anaconda Prompt, puis rouvrir Ubuntu
Cannot connect to the Docker daemon at unix:///var/run/docker.sock	le service Docker n'est pas démarré	sudo systemctl start docker ; s'il échoue, vérifier systemd (ligne plus haut)
Ubuntu s'arrête dès que vous fermez sa fenêtre, et vos conteneurs avec elle	le délai d'inactivité de WSL est au défaut	l'étape 6 ; en attendant, garder une fenêtre Ubuntu ouverte
pyodbc.drivers() ne montre aucun pilote PostgreSQL	psqlODBC n'est pas installé, ou c'est la version 32 bits	installer psqlodbc_x64.msi (étape 7)
'psql' n'est pas reconnu en tant que commande interne	les outils en ligne de commande de PostgreSQL ne sont pas dans le chemin de recherche	la commande PowerShell de l'étape 8, puis rouvrir l'Anaconda Prompt
psql: error: connection to server ... failed: FATAL: password authentication failed	le mot de passe de postgres n'est pas celui choisi à l'installation	le retrouver ; sinon, réinstaller PostgreSQL, ou changer le mot de passe depuis pgAdmin
« Le dossier du projet n'existe pas » au lancement du générateur	l'étape 1 n'est pas faite, ou le dossier porte un autre nom	créer Projet_Boreal, exactement, dans Data_Domain
python: can't open file sur generer_boreal.py	l'Anaconda Prompt n'est pas dans Data_Domain	se placer dans Data_Domain, puis relancer

Annexe — Tout le code, d'un seul tenant

Pour rejouer le socle sur un autre poste sans relire les explications. L'Anaconda Prompt travaille depuis Data_Domain.

1 — Le dossier, Python, Git et DuckDB (Anaconda Prompt)

```
mkdir Projet_Boreal
conda create -n formations python=3.11 -y
conda activate formations
pip install -r 01_Data_Engineering\requirements.txt
python -c "import duckdb, pyodbc, pytest; print(duckdb.__version__)"
git config --global user.name "Votre Nom"
```

```
git config --global user.email "vous@exemple.com"
winget install DuckDB.cli
```

(Après winget, fermer et rouvrir l'Anaconda Prompt, réactiver formations, puis duckdb --version.)

2 — Docker Engine (dans Ubuntu ; wsl -L -v, et au besoin wsl --install -d Ubuntu-26.04 en administrateur)

```
ps -p 1 -o comm=

sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
exit
```

Puis, dans Ubuntu rouvert :

```
docker --version
docker compose version
docker run hello-world
docker ps -a
docker container prune
exit
```

3 — Les pilotes et les moteurs (installer psqlodbc_x64.msi et, pour SQL avancé, PostgreSQL ; puis, dans l'Anaconda Prompt)

```
python -c "import pyodbc; print(pyodbc.drivers())"
psql --version
```

4 — Les données de Boréal (Anaconda Prompt)

```
python _univers/boreal/generer_boreal.py --projet Projet_Boreal
```